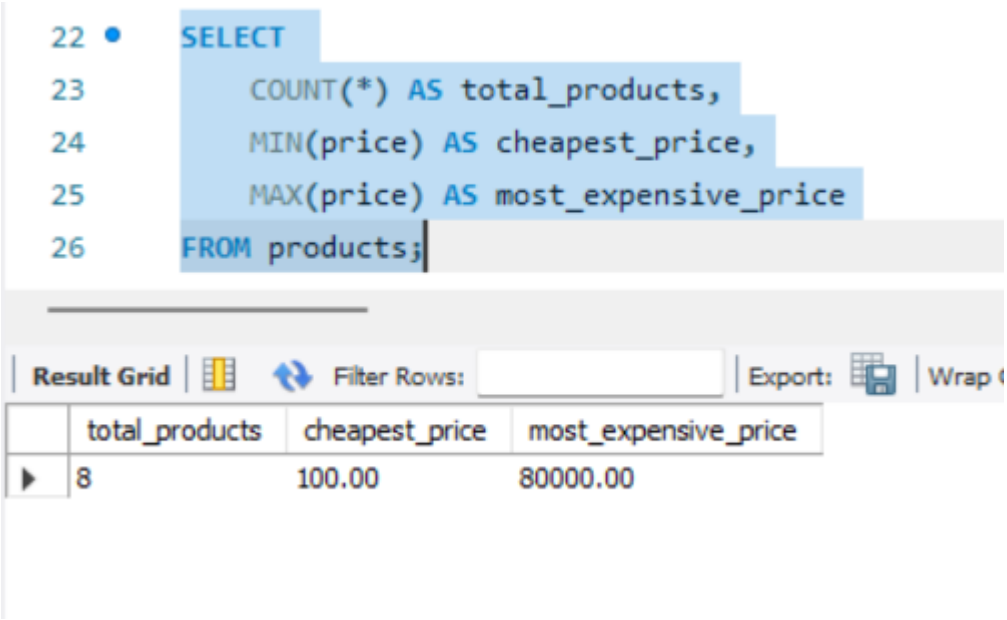
1. Total number of products, cheapest price, and most expensive price in one Query.

```
SELECT
   COUNT(*) AS total_products,
   MIN(price) AS cheapest_price,
   MAX(price) AS most_expensive_price
FROM products;
```




`COUNT()` counts products, `MIN()` finds the lowest price, and `MAX()` finds the highest price.

2. Total stock of all products, and total stock of Electronics only.

```
SELECT
   SUM(stock) AS total_stock,
   SUM(CASE WHEN category = 'Electronics' THEN stock ELSE 0 END)
      AS electronics_stock
FROM products;
```

```
28 •  SELECT
29        SUM(stock) AS total_stock,
30        SUM(CASE
31            WHEN category = 'Electronics' THEN stock
32            ELSE 0
33        END) AS electronics_stock
34    FROM products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

| total_stock | electronics_stock |
|---|---|
| 258 | 85 |

SUM() calculates the total stock of all products and the total stock of Electronics products separately.

3. Average price per category (GROUP BY).

SELECT category, AVG(price) AS average_price
FROM products
GROUP BY category;

```
36 •  SELECT
37        category,
38        AVG(price) AS average_price
39    FROM products
40    GROUP BY category;
```

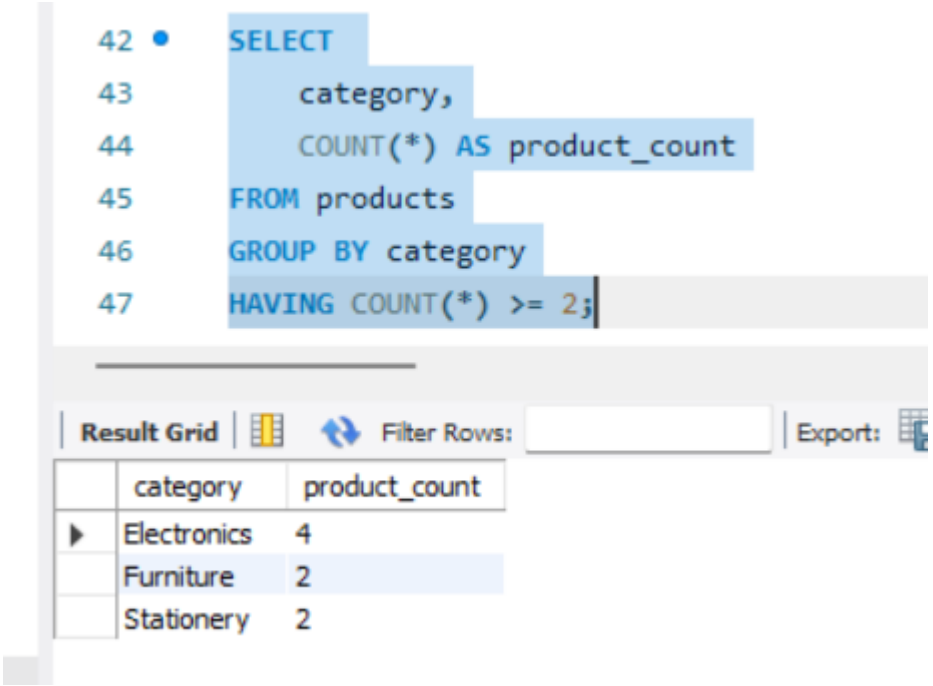
Result Grid | Filter Rows: | Export:

| category | average_price |
|---|---|
| Electronics | 34500.000000 |
| Furniture | 18500.000000 |
| Stationery | 300.000000 |

`GROUP BY` groups products by category, and `AVG()` calculates the average price of each category.

4. Count products per category, but show only categories with at least 2 products. WHERE or HAVING? Why?

SELECT category, COUNT(*) AS total_products
FROM products
GROUP BY category
HAVING COUNT(*) >= 2;




Use `HAVING` because it filters grouped results based on the product count after `GROUP BY`.

5. Average price per category, counting only products cheaper than 60000. Which filter goes in WHERE?

SELECT category, AVG(price) AS average_price
FROM products
WHERE price < 60000
GROUP BY category;

```
48
49 •  SELECT
50        category,
51        AVG(price) AS average_price
52    FROM products
53    WHERE price < 60000
54    GROUP BY category;
```

Result Grid | Filter Rows: | Export:

| category | average_price |
|---|---|
| Electronics | 19333.333333 |
| Furniture | 18500.000000 |
| Stationery | 300.000000 |

Use `WHERE` to filter individual products priced below 60000 before calculating each category's average.

6. Categories where the average price is above 1000, sorted biggest first. Use the full general order.

```
56 •  SELECT
57        category,
58        AVG(price) AS average_price
59    FROM products
60    WHERE price IS NOT NULL
61    GROUP BY category
62    HAVING AVG(price) > 1000
63    ORDER BY average_price DESC;
```

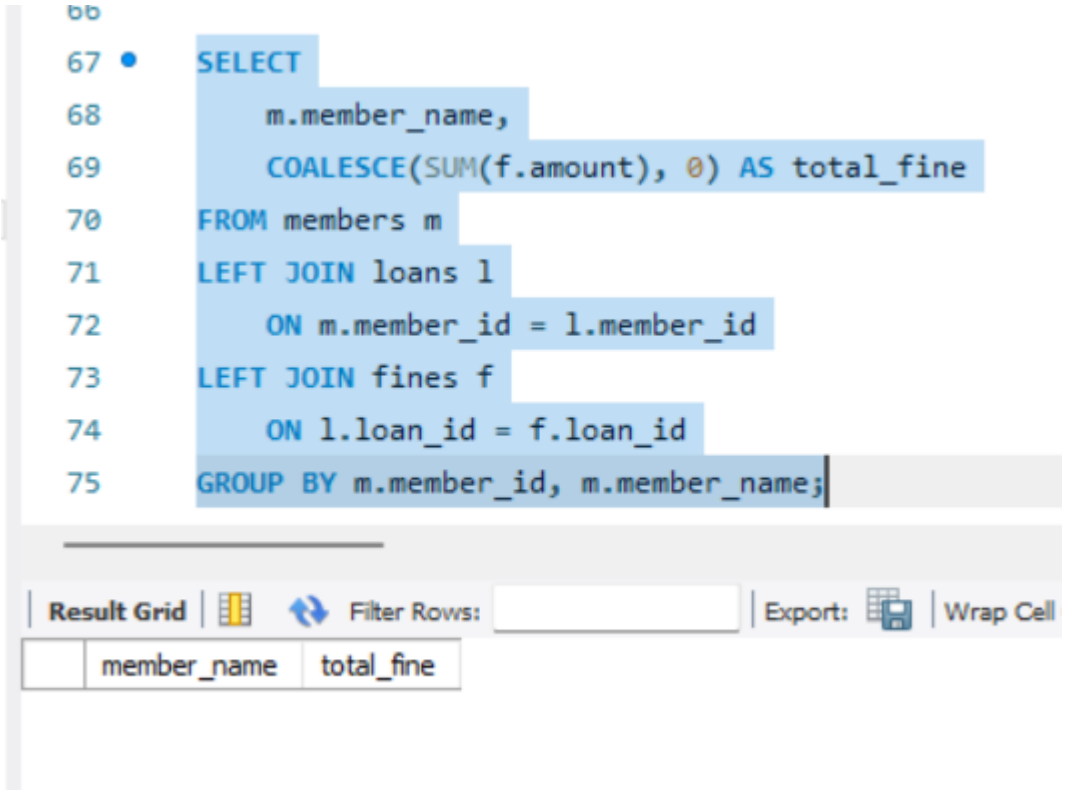
Result Grid | Filter Rows: | Export: | W

| category | average_price |
|---|---|
| Electronics | 34500.000000 |
| Furniture | 18500.000000 |

7. Library: total fine per member (join + GROUP BY). Members with no

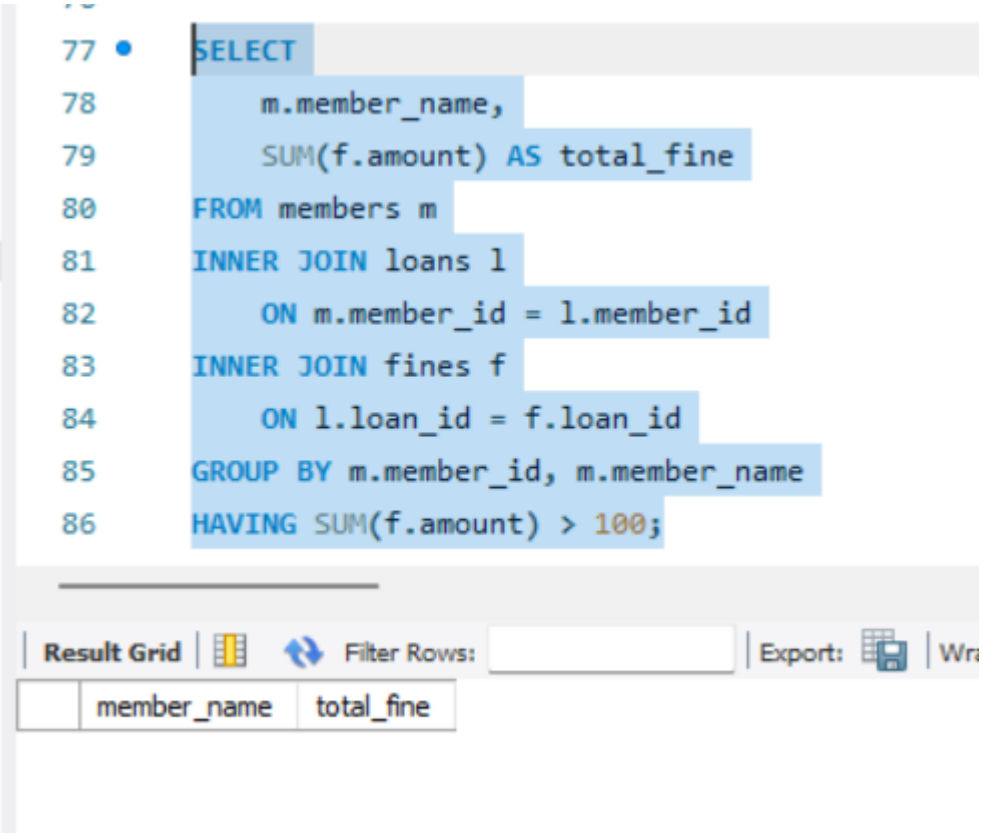
loans must show 0. Which join?

```
SELECT
    m.name,
    COALESCE(SUM(l.fine), 0) AS total_fine
FROM members m
LEFT JOIN loans l
    ON m.member_id = l.member_id
GROUP BY m.member_id, m.name;
```




Use `LEFT JOIN` to include all members, even those without loans. `COALESCE()` displays `0` instead of `NULL` when the total fine is missing.

8. Library: members whose total fine is above 100 (HAVING).




`HAVING SUM(l.fine) > 100` returns only members whose combined fines exceed 100.

9. COUNT(*) vs COUNT(fine): make one fine NULL and show the Difference.

```
UPDATE loans
SET fine = NULL
WHERE loan_id = 1;

SELECT
   COUNT(*) AS total_rows,
   COUNT(fine) AS fines_recorded
FROM loans;
```

`COUNT(*)` counts every row, including rows with `NULL` fines, while `COUNT(fine)` counts only rows where `fine` is not `NULL`.

10. In one sentence: why can HAVING see COUNT(*) but WHERE cannot?

`WHERE` filters individual rows before grouping, while `HAVING` filters grouped results after aggregate functions such as `COUNT(*)` have been calculated.